EXPRESSIVE PYTHON

Applying Python to Educational and Scientific Data

Final Workshop Presenter Script

Mid-Western Educational Research Association (MWERA) 2026

Jason Byhring

Master of Science in Education (M.S.Ed.) in Instructional Technology

RESEARCHER EMPHASIS: This script is grounded in the completed Google Colab notebook. The core analytical sequence is: import an SPSS file, inspect structure and metadata, summarize the data, visualize patterns, test a group-difference question with one-way ANOVA, and model GPA from weekly study hours with ordinary least squares regression.

Core workshop workflow: Import → Inspect → Summarize → Visualize → Test → Model → Interpret

Opening: Position Python as a Research Tool

Good morning, everyone, and thank you for joining me. My name is Jason Byhring, and today our workshop is titled **Expressive Python: Applying Python to Educational and Scientific Data**.

The goal today is not to turn everyone in the room into a software developer. Instead, I want to show how Python can function as a practical research environment - one in which we can move from a real research dataset to inspection, description, visualization, statistical testing, modeling, and interpretation without leaving the same notebook.

As researchers, our questions come first. We ask: What is in the dataset? Are the variables represented correctly? What does the sample look like? Are there meaningful differences among groups? Are two variables related? How much uncertainty is associated with the pattern we observe? Python gives us a reproducible way to investigate those questions.

The sequence I want you to remember is: **Import, Inspect, Summarize, Visualize, Test, Model, and Interpret.** Every library and every line of code we use today fits somewhere in that research workflow.

RESEARCHER EMPHASIS: Programming is the mechanism; research design and statistical reasoning determine what should be done and what the results mean.

The Libraries We Will Actually Use

Our completed notebook uses six major libraries or packages. **Pyreadstat** reads SPSS .sav files and preserves access to SPSS metadata. **Pandas** gives us the DataFrame and tools for inspecting, grouping, sorting, and summarizing data. **NumPy** provides foundational numerical operations. **Matplotlib** creates statistical graphics. **SciPy** supplies scientific and inferential statistical procedures - today, a one-way ANOVA. Finally, **Statsmodels** gives us formal statistical modeling and researcher-friendly regression output.

PRESENTER / DEMO CUE: Point out that these libraries are complementary. Researchers rarely need one library to do everything.

Google Colab and the File Workspace

We are using Google Colab as our notebook environment. Colab lets us combine executable Python code, written explanation, equations, and output in one document. The Files panel is the temporary file workspace associated with the active Colab runtime. When we upload our SPSS .sav file there, Python can read it directly.

Because this is a cloud-based runtime, the Files panel should not be described as permanent local storage. Files uploaded to the runtime are temporary, and normal Colab use requires an internet connection.

PRESENTER / DEMO CUE: Have participants download mwera_student_academic_success_dataset.sav, upload it to the Colab Files panel, and verify that the filename appears before continuing.

Part 1 - Importing SPSS Data with Pyreadstat

We begin with the dataset in its native SPSS format. This matters because an SPSS file can contain more than the rectangular data matrix. It can also contain metadata - including value labels that tell us what numeric codes mean.

```
import pandas as pd
import pyreadstat

df, meta = pyreadstat.read_sav(
    "mwera_student_academic_success_dataset.sav"
)
```

The function **pyreadstat.read_sav()** returns two objects. The first is **df**, our Pandas DataFrame containing the observations and variables. The second is **meta**, an object containing metadata from the SPSS file.

RESEARCHER EMPHASIS: A number without its coding scheme can be ambiguous. Preserving metadata protects the meaning of variables and is especially important when working with categorical survey or administrative data.

Part 2 - Inspect Before You Analyze

Before running a statistical test, I want to know what I actually imported. The first command is deliberately simple.

```
df.head()
```

df.head() shows the first five observations. This is a fast integrity check: Do the columns look right? Did the file load correctly? Are text and numeric fields appearing as expected?

```
df.shape
```

The completed dataset contains **250 observations and 14 variables**. In research terms, shape immediately tells us the scale of the dataset we are about to analyze.

```
df.columns
```

The variables include student ID, age, gender, race/ethnicity, major, GPA, weekly study hours, attendance rate, number of courses, first-generation status, scholarship status, graduation intent, SAT score, and employment status.

RESEARCHER EMPHASIS: Inspection is not busywork. It is part of data validation. A statistical procedure can execute perfectly on the wrong variable or an incorrectly imported dataset.

Inspecting SPSS Metadata

```
_, meta = pyreadstat.read_sav(
    "mwera_student_academic_success_dataset.sav"
)

meta.column_names
meta.variable_value_labels
```

The underscore tells Python that we are intentionally discarding the first returned object and retaining the metadata. In our file, the metadata tells us, for example, that **first_gen**, **scholarship**, and **employed** use zero for No and one for Yes. Graduation intent is coded from 1 through 5, ranging from Very unlikely to Very likely.

RESEARCHER EMPHASIS: The computer can calculate with 0, 1, 2, 3, 4, and 5. The researcher must know what those values represent and whether treating them as categories, ordered categories, or numeric scores is defensible.

Part 3 - Summarizing the Dataset with Pandas

Now that we understand the structure, we can begin describing the sample. Pandas gives us a compact descriptive-statistics table.

```
df.describe()
```

The output includes the count, mean, standard deviation, minimum, quartiles, median, and maximum for numeric variables. In this dataset, mean GPA is approximately **3.163**, mean weekly study time is approximately **21.31 hours**, and mean attendance is approximately **88.08 percent**.

The standard deviation of GPA is approximately **0.441**, while weekly study hours have a standard deviation of approximately **5.868 hours**. These values tell us about dispersion - how much observations vary around their respective means.

RESEARCHER EMPHASIS: Descriptive statistics should generally precede inferential statistics. They provide context for later tests and can expose implausible values, restricted ranges, or unexpected distributions.

Frequency Counts

```
df.value_counts()
df['gpa'].value_counts()
```

Applied to the whole DataFrame, **value_counts()** counts unique row combinations. Because student ID makes each record unique, complete rows appear once. Applied to a single variable, such as GPA, it becomes a frequency table showing how often each observed GPA occurs.

This distinction is useful: the same Pandas method can answer different questions depending on whether we apply it to the entire table or one variable.

Grouping: Split - Apply - Combine

```
mean_gpa = df.groupby('study_hours')['gpa'].mean()
print(mean_gpa)
```

The **groupby()** operation follows a split-apply-combine logic. We split observations into groups, apply a summary operation, and combine the results. Here we calculate mean GPA at each observed level of weekly study hours.

The notebook also groups students by graduation intent and simultaneously calculates mean GPA and mean study hours.

```
intent_comparison = (
    df.groupby('graduation_intent')
      [['gpa', 'study_hours']]
      .mean()
)
display(intent_comparison)
```

In this sample, the pattern rises across the observed graduation-intent categories: students coded 2 average about 2.53 GPA and 15.8 weekly study hours, while students coded 5 average about 3.34 GPA and 23.09 weekly study hours.

RESEARCHER EMPHASIS: This is descriptive evidence, not a causal conclusion. Group means can suggest patterns and motivate questions, but they do not by themselves establish why the groups differ.

Sorting and Standard Deviation

```
df.sort_values(by='study_hours', ascending=False)

df.std(numeric_only=True)
```

Sorting is a simple way to inspect extreme observations. Here it places the highest study-hour observations first. Standard deviation then gives us a numerical measure of spread for each numeric variable.

These are examples of how a researcher can move fluidly between individual cases and aggregate summaries before deciding which inferential analysis is appropriate.

Part 4 - Visualization with Matplotlib

Statistical tables are essential, but they are not always the fastest way to understand a distribution or relationship. Visualization allows us to inspect structure that can be difficult to see in a table.

Matplotlib is the visualization library used in the completed notebook. We begin by comparing weekly study hours across academic majors with a boxplot.

```
groups = df.groupby('major')['study_hours']

plt.boxplot(
    [values for _, values in groups],
    tick_labels=[major for major, _ in groups]
)

plt.xlabel('Major')
plt.ylabel('Study Hours per Week')
plt.xticks(rotation=45)
plt.show()
```

The box represents the middle 50 percent of observations, the line inside the box represents the median, and the whiskers show the spread of non-outlying observations. The purpose is not merely to make the analysis visually appealing. We are looking for differences in center, spread, and potentially unusual observations across majors.

RESEARCHER EMPHASIS: Visualization is part of analysis, not just presentation. It can reveal skew, outliers, nonlinear patterns, unequal variability, or group differences that should influence subsequent statistical decisions.

Visualizing Study Hours and GPA

```
plt.scatter(df['study_hours'], df['gpa'])

local_model = smf.ols(
    'gpa ~ study_hours', data=df
).fit()

x = df['study_hours'].sort_values()
plt.plot(
    x,
    local_model.predict(df.loc[x.index])
)

plt.xlabel('Weekly Study Hours')
plt.ylabel('GPA')
plt.title('Study Hours and GPA')
plt.show()
```

Each point represents a student. Weekly study hours are on the horizontal axis and GPA is on the vertical axis. The fitted line summarizes the estimated linear relationship.

Before reading a regression table, this figure lets us ask whether the association appears positive or negative, whether a linear form seems plausible, how dispersed the observations are around the line, and whether unusual observations deserve attention.

Part 5 - Inferential Testing with SciPy: One-Way ANOVA

Now we move from describing our sample to formal statistical inference. The completed notebook uses SciPy to ask a group-comparison question: **Does mean GPA differ across academic majors?**

A one-way ANOVA is appropriate when we want to compare the means of three or more independent groups on a quantitative outcome. Here, academic major is the categorical grouping variable and GPA is the quantitative outcome.

Our hypotheses are straightforward. The null hypothesis states that mean GPA is equal across the majors represented in the dataset. The alternative states that at least one major has a different mean GPA.

```
from scipy import stats

df_anova, _ = pyreadstat.read_sav(
    "mwera_student_academic_success_dataset.sav"
)

majors = df_anova['major'].unique()

gpa_by_major = [
    df_anova[df_anova['major'] == major]
    ['gpa'].dropna()
    for major in majors
]

f_statistic, p_value = stats.f_oneway(
    *gpa_by_major
)
```

SciPy returns an **F statistic of approximately 1.325** and a **p value of approximately .224**. Using an alpha level of .05, .224 is not below our significance threshold. Therefore, we **fail to reject the null hypothesis**.

For this sample and this analysis, we do not have sufficient statistical evidence to conclude that mean GPA differs across academic majors.

RESEARCHER EMPHASIS: Fail to reject is not the same as proving that all majors have identical GPAs. A non-significant result means the analysis did not provide sufficient evidence against the null hypothesis at the chosen alpha level. Researchers should also consider assumptions, group sizes, statistical power, and effect sizes.

Why the Non-Significant Result Matters

This is an important research lesson. Python is not valuable only when it gives us a statistically significant result. A reproducible analysis that fails to reject the null hypothesis is still evidence. The researcher's responsibility is to report the result accurately rather than search for a test that produces significance.

Part 6 - Simple Linear Regression with Statsmodels

Our final analysis asks a different kind of question: **To what extent do weekly study hours predict college students' current GPA?**

The predictor, X, is weekly study hours. The outcome, Y, is current GPA. We use ordinary least squares, or OLS, regression to estimate the best-fitting linear relationship between them.

The general model is: **predicted Y = b0 + b1X**. In our example: **predicted GPA = intercept + slope times Study Hours**.

OLS chooses the intercept and slope that minimize the sum of squared residuals. A residual is the difference between a student's observed GPA and the GPA predicted by the fitted line.

RESEARCHER EMPHASIS: Regression here supports prediction and association. With this observational dataset, the model does not establish that increasing study time causes GPA to rise.

Revisit the Variables Before Modeling

```
df[['study_hours', 'gpa']].describe()
```

Both variables contain 250 observations. Weekly study hours average 21.306 with a standard deviation of 5.868. GPA averages 3.163 with a standard deviation of 0.441.

```
import numpy as np

np.mean(df['study_hours'])
np.mean(df['gpa'])
```

NumPy provides the underlying numerical operations. In the notebook, it reinforces that the same quantities can be accessed through different layers of the Python scientific ecosystem.

PRESENTER / DEMO CUE: If demonstrating live, point out that the notebook's final displayed NumPy value is GPA because the second expression is the last expression in the cell.

Fit the OLS Model

```
import statsmodels.formula.api as smf

model = smf.ols(
    'gpa ~ study_hours',
    data=df
).fit()

print(model.summary())
```

This line is compact, but it represents the formal statistical model. Statsmodels estimates the intercept, slope, standard errors, t statistics, p values, confidence intervals, R-squared, and additional diagnostics.

Interpreting the Regression Output

Let's interpret the output as researchers rather than simply reading numbers from the table.

The **intercept is approximately 2.506**. Mathematically, this is the predicted GPA when weekly study hours equal zero. Because zero study hours falls outside the observed range of this dataset - the minimum is 6.1 hours - we should be cautious about giving the intercept a strong substantive interpretation.

The **study-hours slope is approximately 0.0309**. Within this fitted model, each additional weekly study hour is associated with an estimated **0.031-point increase in predicted GPA**.

The coefficient has a **p value below .001**. At the conventional alpha level of .05, weekly study hours are a statistically significant predictor of GPA in this model.

The **95 percent confidence interval for the slope is approximately 0.022 to 0.039**. The interval does not include zero, which is consistent with the statistical significance of the coefficient.

The model's **R-squared is .168**. In this sample, the linear model using study hours accounts for approximately **16.8 percent of the observed variance in GPA**. Most of the variability in GPA remains outside this one-predictor model.

The overall model F statistic is approximately **50.18**, with a model probability of approximately **1.44 × 10^-11**, indicating that the fitted model provides statistically significant explanatory information relative to an intercept-only model.

RESEARCHER EMPHASIS: Statistical significance is not the same as practical importance. R-squared, the magnitude of the slope, confidence intervals, study design, measurement quality, assumptions, and substantive theory all matter.

The Fitted Equation

Using the estimated coefficients, our fitted equation is:

Predicted GPA = 2.5056 + 0.0309(Study Hours)

If a student's study hours increase by one unit in the predictor, the model increases predicted GPA by approximately .0309 points. Again, this is an estimated association in this dataset, not a causal guarantee for an individual student.

Residuals and Assumptions

The regression line will not pass through every observation. The vertical difference between an observed GPA and its predicted GPA is the residual. OLS minimizes the sum of squared residuals.

For a full research analysis, we would also evaluate regression assumptions and diagnostics, including linearity, independence of errors, homoscedasticity, and the distribution of residuals. The model summary gives us diagnostic information, but responsible modeling requires more than checking whether one coefficient has a small p value.

RESEARCHER EMPHASIS: The notebook is a workshop demonstration of the workflow. In a publication-quality analysis, assumption checks, theory, sampling design, potential confounders, effect sizes, and model specification deserve explicit attention.

Part 7 - What Each Library Contributes to the Research Process

Library	Workshop role	Researcher role
Pyreadstat	Reads the SPSS .sav file and metadata	Preserve variable meaning and coding information
Pandas	DataFrame inspection, descriptives, grouping, sorting	Understand the sample and prepare defensible analyses
NumPy	Numerical summaries	Perform foundational quantitative operations
Matplotlib	Boxplots and scatterplots	Inspect distributions, groups, relationships, and anomalies
SciPy	One-way ANOVA	Test a formal group-difference hypothesis
Statsmodels	OLS regression and model summary	Estimate, quantify, and interpret a predictive relationship

The Researcher's Role Cannot Be Automated Away

Across this notebook, Python executes our instructions, but it does not determine whether our research question is meaningful, whether a variable validly represents a construct, whether a statistical test is appropriate, whether missing data should be imputed, whether an observed association is practically important, or whether a causal conclusion is justified.

The researcher supplies the conceptual framework. Python supplies a transparent and reproducible computational environment.

That distinction is why I describe Python as expressive. We can express our analytical reasoning in code: how we imported the data, how we inspected it, how we summarized it, what groups we compared, what model we fitted, and how we generated the evidence that supports our interpretation.

A Reproducible Research Mindset

One advantage of a notebook is that the analysis becomes a sequence that another researcher can inspect and rerun. Instead of remembering a series of menu clicks, we retain the commands that generated the result. That does not automatically make the research valid, but it can make the computational process more transparent and reproducible.

RESEARCHER EMPHASIS: Reproducibility is strengthened by preserving the original data, documenting transformations, recording package versions when needed, using clear variable names, commenting analytical decisions, and distinguishing exploratory analysis from confirmatory analysis.

Closing Script

I want to return to the workflow we used today.

Import → Inspect → Summarize → Visualize → Test → Model → Interpret.

We began with an SPSS .sav file and used Pyreadstat to bring both the data and its metadata into Python. We used Pandas to understand the structure of 250 student observations across 14 variables, and to calculate descriptive statistics, frequencies, grouped summaries, sorting, and standard deviations.

We used Matplotlib to make distributions and relationships visible. We used SciPy to test whether mean GPA differed across academic majors, and our result was not statistically significant. Then we used Statsmodels to ask a different question: whether weekly study hours predict GPA. That model identified a statistically significant positive association, with study hours explaining approximately 16.8 percent of the observed variance in GPA.

Those two inferential results are useful together because they remind us that the goal of research is not to manufacture significance. The goal is to use appropriate methods to learn what the evidence supports - and what it does not support.

You do not need to memorize every function we used today. Instead, remember the logic of the workflow. Start with a research question. Understand your data before you model it. Visualize before you over-interpret a coefficient. Choose a statistical procedure that matches the question and the structure of the data. And interpret the result in the context of the research design.

Python can complement the statistical tools many of us already use. More importantly, it gives us a flexible environment in which data, code, statistical reasoning, visualization, and explanation can live together.

Thank you all for your time, participation, and engagement today. I hope this workshop demonstrated that Python can be an accessible tool for educational and scientific research - not only for working with data, but for transforming data into meaningful evidence.

I encourage you to continue experimenting with the techniques we explored and to think about how Python might complement the research methods and statistical tools you already use.

Thank you for joining me, and I hope you enjoy the rest of MWERA.

Final audience prompt: What will you analyze with Python next?

Presenter Quick Reference

Notebook result	Value / interpretation
Dataset	250 observations × 14 variables
Mean GPA	3.16296
Mean weekly study hours	21.3056
ANOVA: GPA by major	F = 1.3251, p = .2244; fail to reject H0
OLS slope: study_hours	b = .0309, p < .001; positive association
OLS intercept	b0 = 2.5056
OLS R-squared	.168; about 16.8% of observed GPA variance
Slope 95% CI	[.022, .039]
Fitted equation	Predicted GPA = 2.5056 + .0309(Study Hours)